# DDS programming workshop

**Mark Jones, G0MGX**

# First things first!

- Please ensure you have registered your car on the parking tablet in the main reception.
- In case of fire alarm, go to the nearest fire exit and make your way to the in/out gates on either the front/rear car parks, as directed.

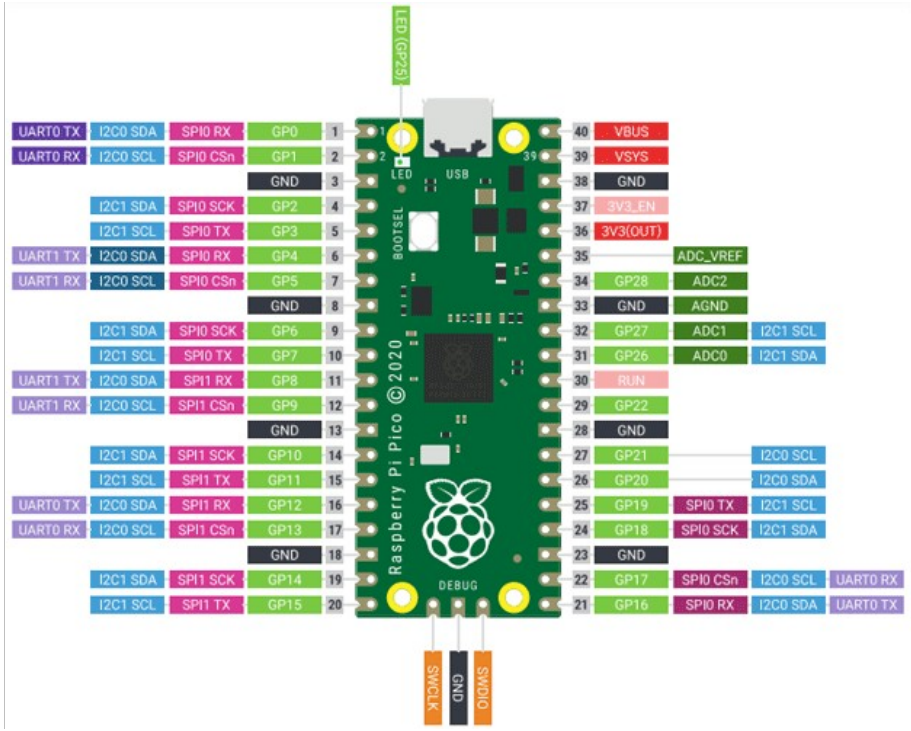
# Resources

- It’s all here:
  - https://drive.google.com/drive/folders/1yQbPeUnxDjYZSZBI3oZwyOzpeQjcCPAF?usp=sharing
- Or maybe here:
  - https://iny.one/k_vgy_
- Source code and board support files are in appropriate folders in this online resource as are datasheets and other gubbins

# Board selection

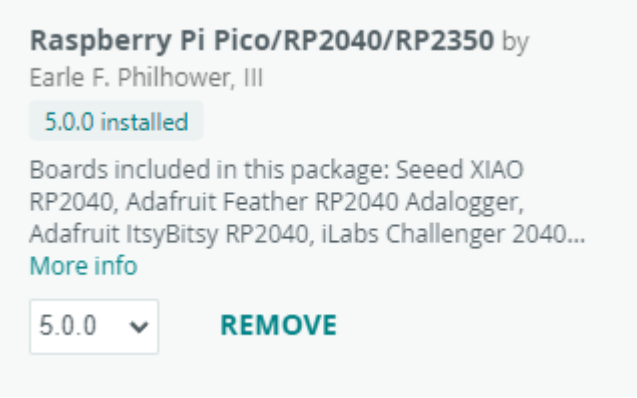
- Primarily based on cost (board cost £5, MCU is 80p), we are using a Pi Pico:
    - RP2340 MCU
    - 32-bit dual core
    - ARM Cortex-M0+ Core @ 133MHz
    - 26 GPIO
    - 2 x 12 bit ADC
    - 2 x SPI
    - 2 x $I^2C$
    - 16 PWM
- It’s no slouch!

# Board selection



# Adding board support

- We need to add board support to the Arduino IDE (which you pre-installed, right?)
- File -> Preferences
  - Add https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json
  - Details in “board support”
- Tools -> Boards -> Boards Manager
  enter “Pi Pico” in the search box and install: Raspberry Pi Pico/RP2040/RP2350



# What’s a DDS?

- A DDS is a signal generator, or Direct Digital Synthesis signal generator
- A device that uses digital techniques to produce an analogue waveform, typically a sine wave, with precise control over frequency and phase
- It achieves this by converting digital data representing the waveform into an analogue signal using a digital-to-analogue converter (DAC)
- DDS generators are known for their accuracy, fine frequency resolution, and ability to quickly switch between frequencies

# The AD9850

- Analog Devices make a large range of DDS chips
- One of the earliest was the AD9850 then AD9851
- Using a 125MHz clock it can generate up to 62.5MHz with a tuning resolution of 0.0291 Hz
- You can generate up to 23 million frequency changes per second (why you would want to I'm not sure)
- You can use a serial or parallel interface; we will look at serial programming

# Frequency calculation

- With the early DDS chips, there is a 40-bit programming register
- This comprises:
  - 32-bit frequency control
  - 5-bit phase control
  - 3-bit control and power down function

# Programming

- To programme the device, we need:
  - Initialisation/Reset
  - Enable output at frequency and phase
- To undertake the three tasks, we need:
  - Pulse digital lines high and low
  - Transfer data a byte at a time

# Programming

- To pulse a data line High and Low we can create a simple routine:

```
void pulseHigh (int pin)
{
  digitalWrite(pin, HIGH);
  delayMicroseconds(2);
  digitalWrite(pin, LOW);
}
```

# Programming

- The datasheet explains that we need to send each byte of data to the DDS one bit at a time, pulsing the CLOCK line after each bit:

```
void transferByte (byte data)
{
  for (int i=0; i<=8; i++, data>>=1)
  {
    digitalWrite(DATA, data & 0x01);
    pulseHigh (CLOCK);
  }
}
```

- There is some fiendishly clever jiggery-pokery going on here, let's look at the detail

# Programming

- We also need to send the phase value to the DDS in the 5th byte of data, using the most significant 5 bits (bits 36 to 40)
- Bit 35 is used for power down for the AD9850 (bits 33 and 34 are always set to 0)

# Programming

- The “data word” is calculated as follows:
- (Target Frequency * 2^32) / Clock Frequency

- On our board the Clock is 125000000
- Example for 10MHz output:

- Data Word = (10000000 * 2^32) / 125000000
- Data Word and Phase Value/Clock Multiplier:

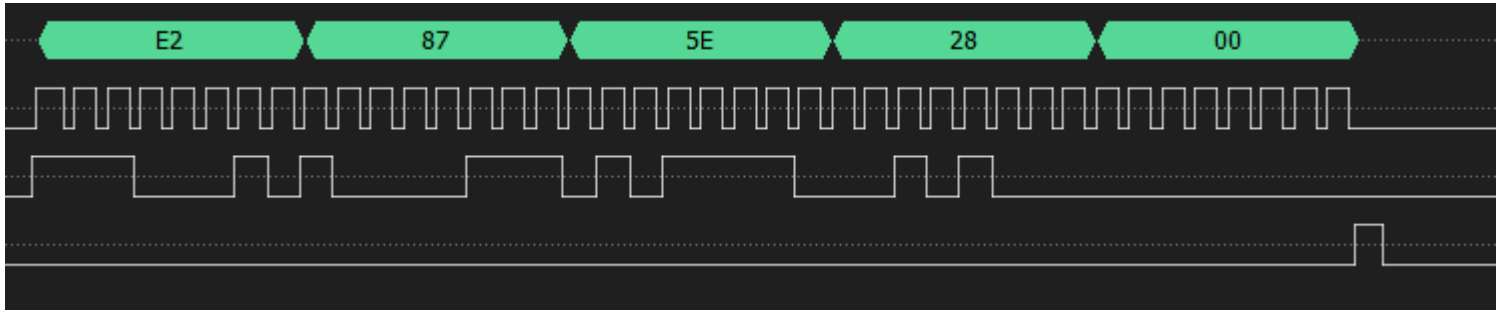
| | | | | | |
|---|---|---|---|---|---|
| E2 | 87 | 5E | 28 | | 0 |
| 11100010 | 10000111 | 01011110 | 00101000 | | 00000000 |

# Programming

- In accordance with the data sheet, we need to:
  - Send the 5 bytes of data to the DDS
    - 32 bits “tuning word”
    - 8 bit “phase and clock multiplier”
  - One bit at a time:
    - Set data line to required bit value
    - Toggle clock line
  - Toggle FQ_UD to complete the action

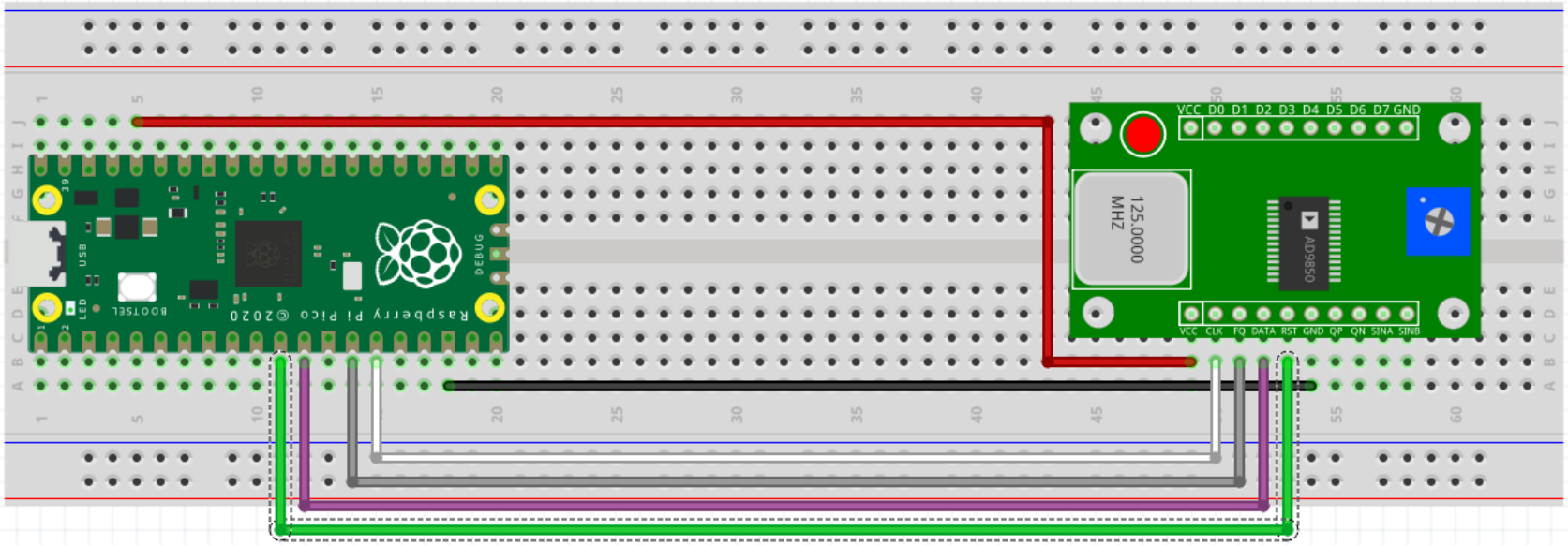
# Programming

- It actually looks like this:




| | | | | | |
|---|---|---|---|---|---|
| | | | | | |
| | | | | | |
| | E2 | 87 | 5E | 28 | | 0 |
| | 11100010 | 10000111 | 01011110 | 00101000 | | 00000000 |

# Board wiring



# The ADF4350

- Analog Devices make a large range of DDS chips – here's another!
- The ADF4350 can generate RF from 137.5MHz to 4.4GHz
- Uses standard SPI interface
- Boards are available from the land where Copyright means "let's copy it right"
    - The newer boards are missing pull-up resistors from the underside of the data lines. Beware!
- On the ADF4350 we need to programme six data registers… it's a little complicated, but the datasheet is your friend

# The ADF4350

- To help with the programming, I have created a spreadsheet with VB macros embedded that will calculate the register values for us

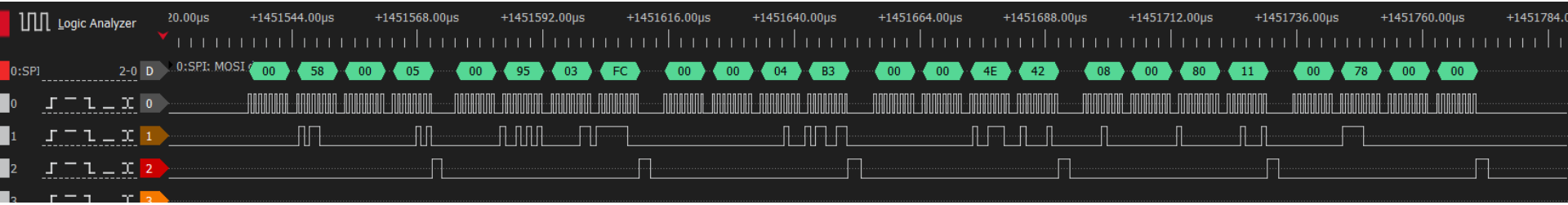
| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | RFout | 1200 | MHz | | | | | | | | | | |
| 2 | Cspacing | 100 | KHz | | Calc | | Enter values in yellow boxes and click Calc | | | | | | |
| 3 | REFin | 10 | MHz | | | | Confim maths by green Check box | | | | | | |
| 4 | | | | | | | | | | | | | |
| 5 | Divider | 2 | | | | | | | | | | | |
| 6 | N | 240 | | | | | | | | | | | |
| 7 | INT | 240 | | | | Register Values | | | | | | | |
| 8 | MOD | 100 | | | Reg0 | 780000 | | | | | | | |
| 9 | FRAC | 0 | | | Reg1 | 8008011 | | | | | | | |
| 10 | | | | | Reg2 | 4E42 | | | | | | | |
| 11 | GCD | 100 | | | Reg3 | 4B3 | | | | | | | |
| 12 | | | | | Reg4 | 9503FC | | | | | | | |
| 13 | MOD | 2 | | | Reg5 | 580005 | | | | | | | |
| 14 | FRAC | 0 | | | | | | | | | | | |
| 15 | | | | | | | | | | | | | |
| 16 | Check | | | | | | | | | | | | |
| 17 | | | | | | | | | | | | | |
| 18 | | | | | | | | | | | | | |
| 19 | | | | | | | | | | | | | |
| 20 | | | | | | | | | | | | | |
| 21 | | | | | | | | | | | | | |

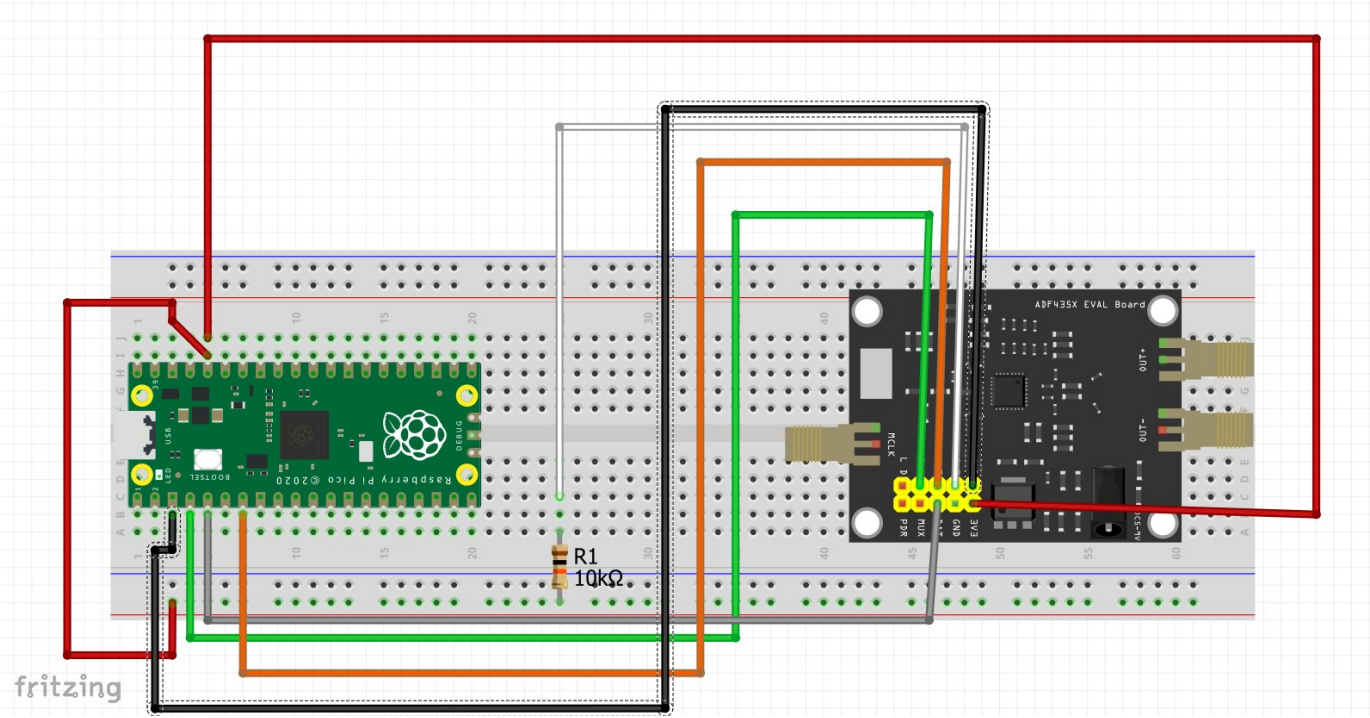
# The ADF4350

- We need to send the 6 register values to the device over the SPI interface.
- In this order reg 5, 4, 3, 2, 1, 0
- Each register comprises 4 lots of 8 bits
- We are sending 192 data bits to the device (6 registers, 32 bits each)
- Its all done using the SPI interface at 10MHz

# Programming

- It actually looks like this (a picture paints a thousand words):



# Board wiring

"Please excuse the crudity of this model. I didn't have time to build it to scale or paint it."